

Theory

Functions

- Functions
- Returning a value from a function
- Sending values to a functions
- Arguments
- External Variables
- Preprocessor directives

Functions

- Avoiding Unnecessary Repetition of Code
- Program Organization

Simple Functions

- Structure of Functions
- The Function Definition
- Calling the Function
- Function Prototype (Declaration)
- Local Variables

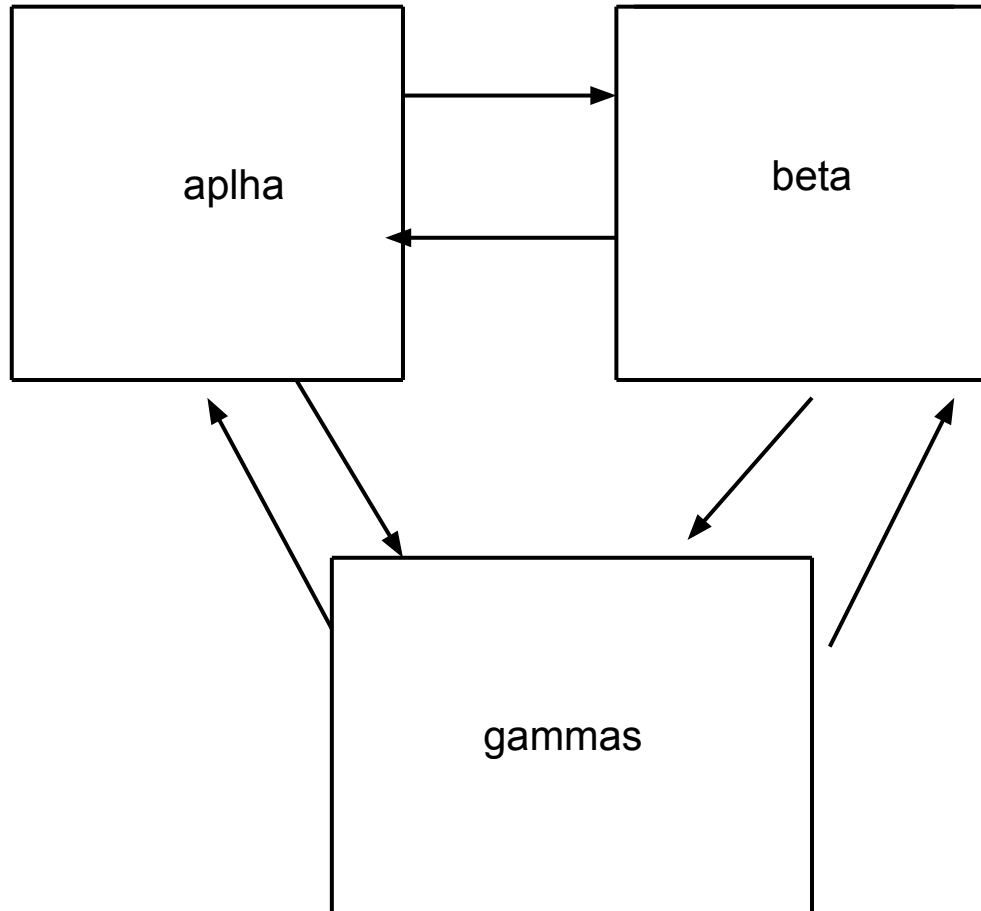
Functions That Return a Value

- The **return** Statement
- Returning Type (int, float, etc)
- Limitation of return()

Using Arguments to Pass Data to a Function

- Structure of Function Call with Arguments
- Passing Variables as Arguments
- Passing Multiple Arguments

Using More Than One Function



All Function
Communicate each
other

Communication Between Functions

External Variables

```
int external_num;
```

```
void print(void);
```

```
void main(void)
```

```
{
```

```
    external_num = 9;
```

```
    print();
```

```
}
```

```
void print(void)
```

```
{
```

```
    printf("%d",external_num);
```

```
}
```


Argument Names in the Declarator

```
void drawSquare(int , int);
```

```
void drawSquare(int width, int height);
```

Preprocessor Directives

The **#define** Directive

```
#define PI3.14159
float area(float);
void main(void)
{
    printf("area of sphere is %.2f", area(3))
}
```

```
float area(float rad)
{
    return (4 * PI * rad * rad);
}
```

why use #define

why not use variable Names

The const Modifier

```
const float PI = 3.141592;
```

Macros

```
#define ERROR printf("\nError.\n");
```

```
#define PR(n) printf("%.2f",n);
```

```
void main(void)
{
    float num1 = 27.25;
    float num2;

    num2 = 1.0 / 3.0
    if(num1 > 2)
    {
        ERROR;
    }
    PR(num1);
    PR(num2);
}
```

Macros and Functions

- `#define PI 3.14159`
- `#define AREA(X) (4* PI * X * X);`
- `#define SUM(x,y) (x+y);`